

Lab 5: FIR Filter -- MMIO vs DMA Streaming

This notebook compares two approaches for running an FIR filter on the Pynq FPGA:

1. **MMIO**: CPU sends one sample at a time via AXI-Lite (slow)
2. **DMA Streaming**: CPU sets up a bulk transfer, HW processes autonomously (fast)

Both use the same 21-tap low-pass filter.

1. Load Overlay

```
In [1]: import time
import struct
import numpy as np
from pynq import Overlay, allocate

ol = Overlay("fir_design.bit")
print("Overlay loaded")
print("IP blocks:", list(ol.ip_dict.keys()))

dma          = ol.axi_dma_0
fir_stream   = ol.fir_stream_0
fir_mmio_ip  = ol.fir_mmio_0
hw_timer     = ol.axi_timer_0
```

```
Overlay loaded
IP blocks: ['fir_stream_0', 'fir_mmio_0', 'axi_timer_0', 'axi_dma_0', 'ps
7']
```

2. Prepare Test Signal

50 Hz (passband) + 300 Hz (stopband), sampled at 1 kHz.

```
In [2]: N_SAMPLES = 256
FS = 1000.0
t = np.arange(N_SAMPLES) / FS
x = (0.7 * np.sin(2 * np.pi * 50.0 * t) +
     0.3 * np.sin(2 * np.pi * 300.0 * t)).astype(np.float32)
x[0] += 1.0

print(f"Input: {N_SAMPLES} samples, dtype={x.dtype}")
```

```
Input: 256 samples, dtype=float32
```

3. MMIO FIR (one sample per round-trip)

Each sample requires: write input → start IP → poll done → read output.

Note: Check the register offsets against the HLS driver header (`xfir_mmio_hw.h`) after synthesis.

```
In [3]: # Register offsets (from HLS synthesis report -- verify after build)
CTRL_REG      = 0x00
X_IN_OFFSET   = 0x18
RETURN_OFFSET = 0x10

def float_to_uint(f):
    return struct.unpack('<I', struct.pack('<f', f))[0]

def uint_to_float(u):
    return struct.unpack('<f', struct.pack('<I', u & 0xFFFFFFFF))[0]

def mmio_fir_one_sample(ip, x_val):
    ip.write(X_IN_OFFSET, float_to_uint(x_val))
    ip.write(CTRL_REG, 0x01) # ap_start
    while (ip.read(CTRL_REG) & 0x02) == 0: # wait ap_done
        pass
    return uint_to_float(ip.read(RETURN_OFFSET))
```

```
In [4]: y_mmio = np.zeros(N_SAMPLES, dtype=np.float32)

t0 = time.perf_counter()
for i in range(N_SAMPLES):
    y_mmio[i] = mmio_fir_one_sample(fir_mmio_ip, float(x[i]))
t_mmio = time.perf_counter() - t0

print(f"MMIO: {N_SAMPLES} samples in {t_mmio*1e3:.2f} ms "
      f"({t_mmio/N_SAMPLES*1e6:.1f} us/sample)")
```

MMIO: 256 samples in 39.90 ms (155.9 us/sample)

4. DMA Streaming FIR

Data path: PS → DMA TX → AXI-Stream → FIR → AXI-Stream → DMA RX → PS

The CPU only sets up the transfer and waits -- the FPGA processes all samples autonomously.

```
In [5]: in_buf = allocate(shape=(N_SAMPLES,), dtype=np.float32)
out_buf = allocate(shape=(N_SAMPLES,), dtype=np.float32)

np.copyto(in_buf, x)
out_buf[:] = 0

# Configure streaming FIR: set sample count and start
# Register 0x10 = 'n' parameter (check synthesis report)
fir_stream.write(0x10, N_SAMPLES)
fir_stream.write(0x00, 0x01) # ap_start

t0 = time.perf_counter()
dma.sendchannel.transfer(in_buf)
dma.recvchannel.transfer(out_buf)
dma.sendchannel.wait()
dma.recvchannel.wait()
t_dma = time.perf_counter() - t0

y_dma = np.array(out_buf, dtype=np.float32)
print(f"DMA: {N_SAMPLES} samples in {t_dma*1e3:.2f} ms "
      f"({t_dma/N_SAMPLES*1e6:.1f} us/sample)")
```

DMA: 256 samples in 3.39 ms (13.3 us/sample)

5. AXI Timer (cycle-accurate HW timing)

The AXI Timer counts PL clock cycles. This measures only the HW processing time (including DMA transfer), without Python overhead.

```
In [6]: TCSR0, TLR0, TCR0 = 0x00, 0x04, 0x08
FCLK_MHZ = 100.0

def timer_start(tmr):
    tmr.write(TLR0, 0)
    tmr.write(TCSR0, 0x020) # load
    tmr.write(TCSR0, 0x080) # enable, count up

def timer_stop(tmr):
    cycles = tmr.read(TCR0)
    tmr.write(TCSR0, 0x000)
    return cycles
```

```
In [7]: # Re-run DMA test with HW timer
np.copyto(in_buf, x)
out_buf[:] = 0

fir_stream.write(0x10, N_SAMPLES)
fir_stream.write(0x00, 0x01)

timer_start(hw_timer)
dma.sendchannel.transfer(in_buf)
dma.recvchannel.transfer(out_buf)
dma.sendchannel.wait()
dma.recvchannel.wait()
cycles = timer_stop(hw_timer)

print(f"HW timer: {cycles} cycles = {cycles/FCLK_MHZ:.1f} us @ {FCLK_MHZ:.1f} MHz")
```

HW timer: 631552 cycles = 6315.5 us @ 100 MHz

6. Comparison

In [8]:

```
print(f"{'Method':<12} {'Time (ms)':>10} {'Speedup':>10}")
print("-" * 34)
print(f"{'MMIO':<12} {t_mmio*1e3:>10.2f} {'1.0x':>10}")
print(f"{'DMA':<12} {t_dma*1e3:>10.2f} {t_mmio/t_dma:>9.1f}x")
print(f"{'HW cycles':<12} {cycles/FCLK_MHZ/1e3:>10.3f} {'(PL only)':>10}")
print()

max_diff = np.max(np.abs(y_dma - y_mmio))
print(f"Max |DMA - MMIO| = {max_diff:.2e}")
assert max_diff < 1e-4, "Output mismatch!"
```

Method	Time (ms)	Speedup
MMIO	39.90	1.0x
DMA	3.39	11.8x
HW cycles	6.316	(PL only)

Max |DMA - MMIO| = 0.00e+00

7. Plot Results

In [9]:

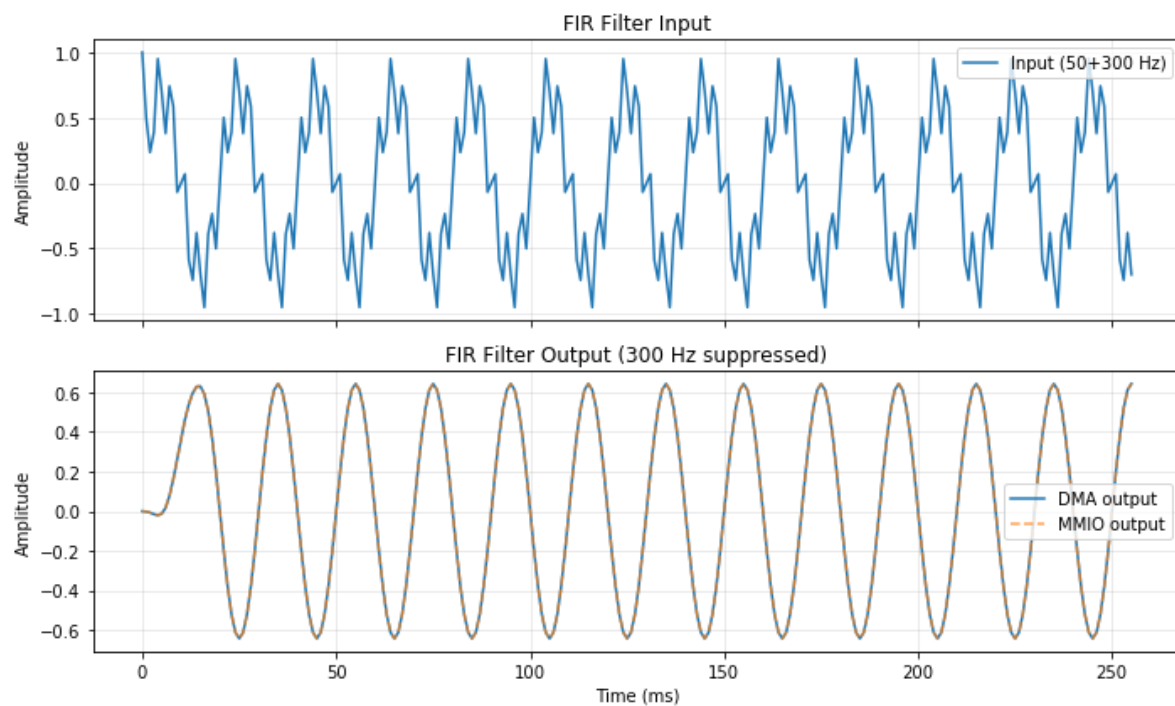
```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 1, figsize=(10, 6), sharex=True)

axes[0].plot(t * 1e3, x, label="Input (50+300 Hz)")
axes[0].set_ylabel("Amplitude")
axes[0].set_title("FIR Filter Input")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(t * 1e3, y_dma, label="DMA output")
axes[1].plot(t * 1e3, y_mmio, '--', label="MMIO output", alpha=0.7)
axes[1].set_xlabel("Time (ms)")
axes[1].set_ylabel("Amplitude")
axes[1].set_title("FIR Filter Output (300 Hz suppressed)")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



8. Cleanup

```
In [ ]: in_buf.freebuffer()
        out_buf.freebuffer()
```